INTRODUCCIÓN A LA PROGRAMACIÓN

Algoritmos y Lógica Básica

Apuntes – Clase 1

Curso de Introducción a la Programación

1. ¿Qué es la programación?

1.1. Definición

Programar es el proceso de diseñar, escribir, probar y mantener un conjunto de instrucciones (código) que le indican a un ordenador cómo resolver un problema o realizar una tarea. Es, en esencia, una forma de comunicación entre humanos y máquinas.

Un programa no es más que una secuencia ordenada de instrucciones escritas en un lenguaje que el ordenador puede interpretar o compilar para ejecutar acciones concretas.

> 💡 **Analogía clave**
> Programar es como escribir una receta de cocina muy detallada para alguien que sigue las instrucciones al pie de la letra, sin asumir nada por su cuenta. Si olvidas un paso o lo describes de forma ambigua, el resultado no será el esperado.

1.2. ¿Por qué aprender a programar?

- Resolución de problemas: programar entrena el pensamiento lógico y estructurado, habilidades transferibles a cualquier disciplina.
- Automatización: permite delegar tareas repetitivas al ordenador, ahorrando tiempo y reduciendo errores humanos.
- Creatividad: con programación se pueden crear desde aplicaciones web hasta videojuegos, pasando por análisis de datos o arte generativo.
- Demanda laboral: el conocimiento de programación es cada vez más valorado en prácticamente todos los sectores profesionales.

1.3. Conceptos fundamentales previos

Hardware vs. Software

Hardware es la parte física del ordenador (procesador, memoria RAM, disco duro, etc.). **Software** es el conjunto de programas e instrucciones que se ejecutan sobre ese hardware. Cuando programamos, creamos software.

Lenguajes de programación

Un lenguaje de programación es un conjunto de reglas sintácticas y semánticas que nos permiten escribir instrucciones comprensibles para el ordenador. Existen cientos de lenguajes, cada uno con sus fortalezas:

- **Bajo nivel**: cercanos al hardware (ensamblador). Control total, pero difíciles de escribir y leer.
- **Alto nivel**: cercanos al lenguaje humano (Python, JavaScript, Java). Más fáciles de aprender y usar.

Compilación vs. Interpretación

Un **compilador** traduce todo el código fuente a lenguaje máquina de una sola vez, generando un ejecutable (ejemplo: C, C++). Un **intérprete** traduce y ejecuta el código línea a línea en tiempo real (ejemplo: Python, JavaScript).

Algunos lenguajes combinan ambos enfoques. Java, por ejemplo, compila a un código intermedio (bytecode) que luego se interpreta en la máquina virtual de Java (JVM).

2. Algoritmos

2.1. ¿Qué es un algoritmo?

Un algoritmo es una secuencia finita, ordenada y no ambigua de pasos que resuelve un problema o realiza una tarea. Los algoritmos existen independientemente de la programación: seguimos algoritmos cuando cocinamos, cuando resolvemos una ecuación o cuando buscamos una palabra en el diccionario.

2.2. Características de un buen algoritmo

1. **Precisión:** cada paso debe estar definido de forma exacta, sin lugar a interpretaciones.
2. **Finitud:** debe terminar tras un número finito de pasos. Un proceso que no termina nunca no es un algoritmo.
3. **Definición de entradas y salidas:** debe especificar qué datos recibe y qué resultado produce.
4. **Efectividad:** cada paso debe ser suficientemente básico como para poder ejecutarse de forma mecánica.
5. **Generalidad:** idealmente resuelve una clase de problemas, no solo un caso particular.

📋 Ejemplo cotidiano: Hacer café con cafetera italiana

1. Desenroscar la cafetera y separar las tres partes.
2. Llenar la base con agua hasta la válvula de seguridad.
3. Colocar el filtro y llenarlo de café molido sin apretar.
4. Enroscar la parte superior sobre la base.
5. Poner la cafetera a fuego medio.
6. Esperar hasta que el café suba y empiece a borbotear.
7. Retirar del fuego y servir.

Este proceso cumple las características: es preciso, finito, tiene una entrada (agua + café) y una salida (café listo), y cada paso es ejecutable.

2.3. Representación de algoritmos

Antes de escribir código, conviene representar el algoritmo de forma visual o textual. Las dos formas más habituales son:

Pseudocódigo

Es una descripción del algoritmo usando un lenguaje informal pero estructurado, cercano al lenguaje natural pero con la organización de un lenguaje de programación. No hay un estándar único; lo importante es que sea claro.

```
ALGORITMO: Determinar si un número es par o impar

INICIO
    LEER número
    SI número MOD 2 == 0 ENTONCES
        ESCRIBIR "El número es par"
    SI NO
        ESCRIBIR "El número es impar"
    FIN SI
FIN
```

Diagramas de flujo

Son representaciones gráficas del algoritmo usando símbolos estandarizados conectados por flechas que indican el flujo de ejecución:

Símbolo	Significado
Óvalo / Círculo	Inicio o Fin del algoritmo
Rectángulo	Proceso o acción (asignación, cálculo)
Rombo	Decisión (condición que se evalúa a verdadero/falso)
Paralelogramo	Entrada o salida de datos
Flechas	Dirección del flujo de ejecución

2.4. Ejemplo completo: calcular el mayor de tres números

Veamos un ejemplo algo más elaborado que combina varias decisiones:

```
ALGORITMO: Mayor de tres números

INICIO
    LEER a, b, c
    SI a >= b Y a >= c ENTONCES
        mayor ← a
    SI NO SI b >= a Y b >= c ENTONCES
        mayor ← b
    SI NO
        mayor ← c
    FIN SI
    ESCRIBIR "El mayor es: ", mayor
FIN
```

Este algoritmo recibe tres números como entrada, los compara entre sí y devuelve el mayor. Utiliza estructuras de decisión encadenadas, algo que veremos en detalle más adelante.

3. Lógica básica de programación

3.1. Variables y tipos de datos

Una variable es un espacio en la memoria del ordenador al que le asignamos un nombre y en el que almacenamos un valor. Podemos pensar en ella como una caja etiquetada donde guardamos información.

```
nombre ← "Ana"          // Variable de tipo texto (cadena / string)
edad ← 25                // Variable de tipo número entero
altura ← 1.68            // Variable de tipo número decimal (flotante)
aprobado ← VERDADERO     // Variable de tipo lógico (booleano)
```

Tipos de datos fundamentales

Tipo	Ejemplo	Descripción
Entero	42, -7, 0	Números sin parte decimal
Flotante	3.14, -0.5	Números con parte decimal
Cadena	"Hola", "123"	Texto entre comillas
Booleano	VERDADERO, FALSO	Solo dos valores posibles

3.2. Operadores

Operadores aritméticos

Permiten realizar cálculos matemáticos básicos:

Operador	Símbolo	Ejemplo
Suma	+	5 + 3 = 8
Resta	-	10 - 4 = 6
Multiplicación	*	3 * 7 = 21
División	/	15 / 4 = 3.75
Módulo	MOD (%)	15 MOD 4 = 3 (resto de la división)

Operadores de comparación (relacionales)

Comparan dos valores y devuelven un resultado booleano (verdadero o falso):

Operador	Símbolo	Ejemplo
Igual a	==	5 == 5 → VERDADERO
Distinto de	!=	5 != 3 → VERDADERO

Mayor que	>	7 > 3 → VERDADERO
Menor que	<	2 < 8 → VERDADERO
Mayor o igual	>=	5 >= 5 → VERDADERO
Menor o igual	<=	3 <= 1 → FALSO

Operadores lógicos

Combinan expresiones booleanas para formar condiciones más complejas:

- **Y (AND):** verdadero solo si ambas condiciones son verdaderas. Ejemplo: (edad > 18 Y tiene_carnet == VERDADERO).
- **O (OR):** verdadero si al menos una condición es verdadera. Ejemplo: (es_festivo O es_fin_de_semana).
- **NO (NOT):** invierte el valor lógico. Ejemplo: NO (llueve) → si llueve es VERDADERO, el resultado es FALSO.

Tabla de verdad resumida

A Y B: solo VERDADERO cuando A = V y B = V
A O B: FALSO únicamente cuando A = F y B = F
NO A: invierte el valor de A

4. Estructuras de control

Las estructuras de control determinan el orden en que se ejecutan las instrucciones de un programa. Son la base que da “inteligencia” a nuestros algoritmos.

4.1. Estructura secuencial

Es la más simple: las instrucciones se ejecutan una detrás de otra, en el orden en que están escritas. Todo programa sigue una estructura secuencial como base.

```
INICIO
    LEER radio
    area ← 3.14159 * radio * radio
    ESCRIBIR "El área del círculo es: ", area
FIN
```

4.2. Estructura condicional (selección)

Permite que el programa tome decisiones y ejecute diferentes bloques de código según se cumpla o no una condición.

Condicional simple (SI – ENTONCES)

```
SI condición ENTONCES
    instrucciones
FIN SI
```

Condicional doble (SI – ENTONCES – SI NO)

```
SI condición ENTONCES
    instrucciones_si_verdadero
SI NO
    instrucciones_si_falso
FIN SI
```

Condicional múltiple (SI encadenados)

```
SI condición_1 ENTONCES
    instrucciones_1
SI NO SI condición_2 ENTONCES
    instrucciones_2
SI NO SI condición_3 ENTONCES
    instrucciones_3
SI NO
    instrucciones_por_defecto
```

```
FIN SI
```

Ejemplo práctico: clasificar una nota

```
ALGORITMO: Clasificar nota

INICIO
    LEER nota
    SI nota >= 9 ENTONCES
        ESCRIBIR "Sobresaliente"
    SI NO SI nota >= 7 ENTONCES
        ESCRIBIR "Notable"
    SI NO SI nota >= 5 ENTONCES
        ESCRIBIR "Aprobado"
    SI NO
        ESCRIBIR "Suspenso"
    FIN SI
FIN
```

4.3. Estructuras repetitivas (bucles)

Los bucles permiten repetir un bloque de instrucciones mientras se cumpla una condición, o un número determinado de veces. Son fundamentales para evitar escribir código repetitivo.

Bucle MIENTRAS (while)

Repite las instrucciones mientras la condición sea verdadera. La condición se evalúa antes de cada repetición, por lo que es posible que el cuerpo no se ejecute ninguna vez.

```
MIENTRAS condición HACER
    instrucciones
FIN MIENTRAS
```

Ejemplo: cuenta atrás

```
INICIO
    contador ← 10
    MIENTRAS contador > 0 HACER
        ESCRIBIR contador
        contador ← contador - 1
    FIN MIENTRAS
    ESCRIBIR "¡Despegue!"
FIN
```

Bucle PARA (for)

Se usa cuando sabemos de antemano cuántas veces queremos repetir el bloque. Un contador se incrementa automáticamente en cada iteración.

```
PARA i DESDE 1 HASTA 10 HACER
    ESCRIBIR i
FIN PARA
```

Bucle REPETIR – HASTA (do – while)

Similar al bucle MIENTRAS, pero la condición se evalúa después de ejecutar el bloque, lo que garantiza que el cuerpo se ejecuta al menos una vez.

```
REPETIR
    LEER contraseña
HASTA QUE contraseña == "secreta"
```

⚠ Cuidado: bucles infinitos

Si la condición de un bucle MIENTRAS nunca se hace falsa, el programa se queda ejecutándose indefinidamente. Esto se llama bucle infinito y es uno de los errores más comunes al empezar a programar.

Consejo: asegúrate siempre de que algo dentro del bucle modifique la condición para que eventualmente deje de cumplirse.

5. Ejercicios resueltos

Ejercicio 1: Suma de los N primeros números naturales

Enunciado: Escribir un algoritmo que, dado un número N introducido por el usuario, calcule la suma de todos los números naturales desde 1 hasta N.

```
ALGORITMO: Suma de 1 a N

INICIO
    LEER N
    suma ← 0
    PARA i DESDE 1 HASTA N HACER
        suma ← suma + i
    FIN PARA
    ESCRIBIR "La suma es: ", suma
FIN
```

Traza con N = 5: suma va tomando los valores 0 → 1 → 3 → 6 → 10 → 15. Resultado: 15.

Ejercicio 2: Determinar si un número es primo

Enunciado: Dado un número entero mayor que 1, determinar si es primo (solo divisible por 1 y por sí mismo).

```
ALGORITMO: Es primo

INICIO
    LEER n
    es_primo ← VERDADERO
    SI n <= 1 ENTONCES
        es_primo ← FALSO
    SI NO
        PARA i DESDE 2 HASTA n - 1 HACER
            SI n MOD i == 0 ENTONCES
                es_primo ← FALSO
            FIN SI
        FIN PARA
    FIN SI
    SI es_primo ENTONCES
        ESCRIBIR n, " es primo"
    SI NO
        ESCRIBIR n, " no es primo"
    FIN SI
FIN
```

Nota: este algoritmo puede optimizarse comprobando divisores solo hasta la raíz cuadrada de n, pero para nivel introductorio esta versión es más clara.

Ejercicio 3: Adivina el número

Enunciado: El programa “piensa” un número secreto y el usuario debe adivinarlo. El programa indica si el intento es demasiado alto, demasiado bajo o correcto.

```
ALGORITMO: Adivina el número

INICIO
    secreto ← 42
    adivinado ← FALSO
    intentos ← 0

    MIENTRAS adivinado == FALSO HACER
        ESCRIBIR "Introduce tu intento: "
        LEER intento
        intentos ← intentos + 1

        SI intento == secreto ENTONCES
            adivinado ← VERDADERO
            ESCRIBIR "¡Correcto! Lo adivinaste en ", intentos, "
intentos"
        SI NO SI intento > secreto ENTONCES
            ESCRIBIR "Demasiado alto"
        SI NO
            ESCRIBIR "Demasiado bajo"
        FIN SI
    FIN MIENTRAS
FIN
```

Este ejercicio combina un bucle MIENTRAS con condicionales encadenados y un contador de intentos. Es un patrón muy común en programación.

6. Ejercicios propuestos

Intenta resolver estos ejercicios escribiendo el pseudocódigo antes de consultar la solución. Recuerda: no existe una única solución correcta; lo importante es que el algoritmo sea claro, finito y produzca el resultado esperado.

1. **Tabla de multiplicar:** pedir un número al usuario y mostrar su tabla de multiplicar del 1 al 10.
2. **Factorial:** dado un número N, calcular N! (N factorial = N × (N-1) × ... × 2 × 1).
3. **Conversor de temperatura:** pedir una temperatura en Celsius y mostrar el equivalente en Fahrenheit (F = C × 9/5 + 32). Añadir una opción para hacer la conversión inversa.
4. **Número invertido:** dado un número entero positivo, mostrar sus dígitos en orden inverso (ejemplo: 1234 → 4321). Pista: usar MOD 10 y división entera.
5. **Validación de contraseña:** pedir al usuario que introduzca una contraseña. Si no coincide con la almacenada, permitir hasta 3 intentos antes de bloquear el acceso.
6. **Fibonacci:** mostrar los primeros N términos de la sucesión de Fibonacci (0, 1, 1, 2, 3, 5, 8, 13...).

7. Resumen de la clase

Concepto	Idea clave
Programación	Comunicar instrucciones precisas a un ordenador
Algoritmo	Secuencia finita, ordenada y no ambigua de pasos
Variables	Espacios de memoria con nombre para almacenar datos
Tipos de datos	Entero, flotante, cadena, booleano
Operadores	Aritméticos, de comparación y lógicos
Secuencia	Instrucciones ejecutadas en orden lineal
Condicional	Ejecutar código según se cumpla una condición
Bucles	Repetir instrucciones (MIENTRAS, PARA, REPETIR)
Pseudocódigo	Representación textual informal de un algoritmo
Diagrama de flujo	Representación gráfica con símbolos estándar

8. Inteligencia Artificial en 2026: contexto y uso como tutor

8.1. Breve cronología de la IA (2022–2026)

La inteligencia artificial ha experimentado una aceleración sin precedentes en los últimos cuatro años. Este timeline esquemático recoge los hitos más relevantes:

Año	Hitos principales
2022	DALL·E 2 y Stable Diffusion popularizan la generación de imágenes con IA. En noviembre, OpenAI lanza ChatGPT (basado en GPT-3.5) y se convierte en fenómeno viral. GitHub Copilot llega como asistente de código en tiempo real.
2023	GPT-4 eleva el nivel de razonamiento y capacidad multimodal (texto + imagen). Anthropic lanza Claude; Google lanza Bard (luego Gemini). Meta libera LLaMA como modelo de código abierto. La IA entra en el aula: ~40% de centros educativos comienzan a integrar herramientas de IA.
2024	Modelos más potentes y especializados: Claude 3, Gemini 1.5 Pro, GPT-4o. Los modelos de razonamiento (o1, o3) permiten resolver problemas matemáticos y científicos complejos paso a paso. La UE comienza a aplicar el AI Act, la primera legislación integral sobre IA.
2025	Generación de vídeo (Sora, Veo), agentes autónomos de código (Codex, Jules), y modelos de razonamiento profundo. DeepSeek-R1 desde China sacude el mercado. GPT-5 y Claude 4 elevan la frontera. La IA pasa de “experimentación” a “integración real” en empresas.
2026	La IA se consolida como herramienta cotidiana. Los asistentes de código entienden el contexto completo de proyectos. Modelos más eficientes y especializados. La adopción se generaliza en educación, salud y desarrollo de software. El foco pasa de “¿qué puede hacer la IA?” a “¿cómo usarla bien?”.

8.2. La IA como tutor de programación

Una de las aplicaciones más transformadoras de la IA actual no es que programe por nosotros, sino que nos ayude a aprender a programar de forma más eficiente. La idea fundamental es usar la IA como tutor personal, no como solucionador automático.

🎯 **Principio clave**

La IA no debe escribir el código por ti. Debe ayudarte a entender por qué tu código funciona (o no funciona), explicarte conceptos cuando te atascas, y sugerirte caminos cuando no sabes por dónde seguir. El aprendizaje real ocurre cuando tú escribes, tú piensas y tú resuelves.

¿Qué puede hacer un tutor de IA por ti?

- **Explicar conceptos:** "¿Qué diferencia hay entre un bucle for y un bucle while en Python?" — La IA puede dártelo con ejemplos adaptados a tu nivel.
- **Revisar tu código:** pega tu código y pregúntale "¿qué estoy haciendo mal?" o "¿cómo puedo mejorar esto?". No le pidas que lo reescriba entero, sino que señale el problema.
- **Generar ejercicios:** "Dame 5 ejercicios de dificultad media sobre listas en Python". La IA puede crear prácticas personalizadas y luego corregirte.
- **Explicar errores:** copia un mensaje de error y pregúntale qué significa y cómo solucionarlo. Esto es mucho más eficiente que buscar en foros.
- **Automatizar lo repetitivo:** tareas mecánicas como formatear datos, generar plantillas básicas o documentar funciones son ideales para delegar a la IA.

¿Qué NO deberías pedirle a la IA?

- **Que resuelva los ejercicios por ti:** si copias la solución directamente, no estás aprendiendo. Es como copiar los deberes de un compañero.
- **Que sea tu única fuente de verdad:** los modelos de IA pueden equivocarse (lo que se llama "alucinar"). Siempre contrasta con la documentación oficial.
- **Que piense por ti:** antes de preguntar, intenta resolverlo tú. Luego usa la IA para verificar o desbloquear.

8.3. Flujo de trabajo recomendado con IA

Cuando te enfrentes a un problema de programación en Python, sigue este proceso:

1. **Lee y comprende el problema.** Dedícale unos minutos a pensar qué se pide, qué datos tienes y qué resultado se espera.
2. **Escribe el pseudocódigo.** Antes de abrir Python, planifica tu solución en papel o pseudocódigo (como hemos aprendido en esta clase).
3. **Inténtalo tú primero.** Escribe tu código en Python, aunque esté incompleto o tenga errores. El intento es donde ocurre el aprendizaje.
4. **Usa la IA como tutor.** Si te atascas, pregúntale algo específico: "Mi bucle no termina, ¿qué puede estar pasando?" en lugar de "Resuélveme el ejercicio".
5. **Verifica y consolida.** Ejecuta tu código, comprueba que funciona con diferentes entradas, y asegúrate de que entiendes cada línea.

8.4. Ejemplo práctico: usar la IA como tutor en Python

Imagina que estás intentando escribir una función en Python que determine si un número es primo (el ejercicio 2 de la sección anterior). Has escrito esto:

```
def es_primo(n):
    for i in range(n):
        if n % i == 0:
            return False
    return True
```

Al ejecutarlo, obtienes un error: ZeroDivisionError. En lugar de pedirle a la IA que te reescriba la función, podrías preguntar:

> 💬 **Ejemplo de prompt al tutor IA**
> «Tengo esta función en Python para comprobar si un número es primo, pero me da un ZeroDivisionError. ¿Puedes explicarme por qué ocurre y darme una pista para arreglarlo sin darme la solución completa?»

La IA te explicaría que range(n) empieza en 0, y dividir entre 0 causa el error. Te sugeriría que pienses desde qué número debería empezar el bucle. Con esa pista, tú ajustas range(2, n) y el código funciona. Has aprendido algo, no solo copiado una solución.

8.5. Herramientas de IA disponibles en abril de 2026

El ecosistema de herramientas de IA útiles para aprender programación es muy amplio. Estas son las principales categorías:

Categoría	Ejemplos	Uso recomendado
Chatbots generalistas	ChatGPT, Claude, Gemini	Explicar conceptos, revisar código, generar ejercicios, resolver dudas puntuales
Asistentes de código	GitHub Copilot, Cursor	Autocompletar código en el editor, sugerir funciones mientras escribes
Tutores especializados	Python Tutor, DeepLearning.AI	Visualizar la ejecución paso a paso, cursos guiados con feedback de IA
Buscadores con IA	Perplexity, Phind	Buscar documentación técnica con respuestas contextualizadas y fuentes

Para este curso, la recomendación es usar un chatbot generalista (ChatGPT, Claude o Gemini, todos tienen versión gratuita) como tutor principal, y Python Tutor (pythontutor.com) para visualizar cómo se ejecuta tu código paso a paso.

> 📖 **Resumen: la IA en tu aprendizaje de programación**
> • Úsala como TUTOR, no como solucionador.
> • Pregunta cosas específicas, no “hazme el ejercicio”.

- Siempre intenta primero tú, luego consulta.
- La IA se equivoca: verifica siempre con la documentación oficial de Python (docs.python.org).
- Automatiza lo repetitivo, pero nunca el pensamiento.